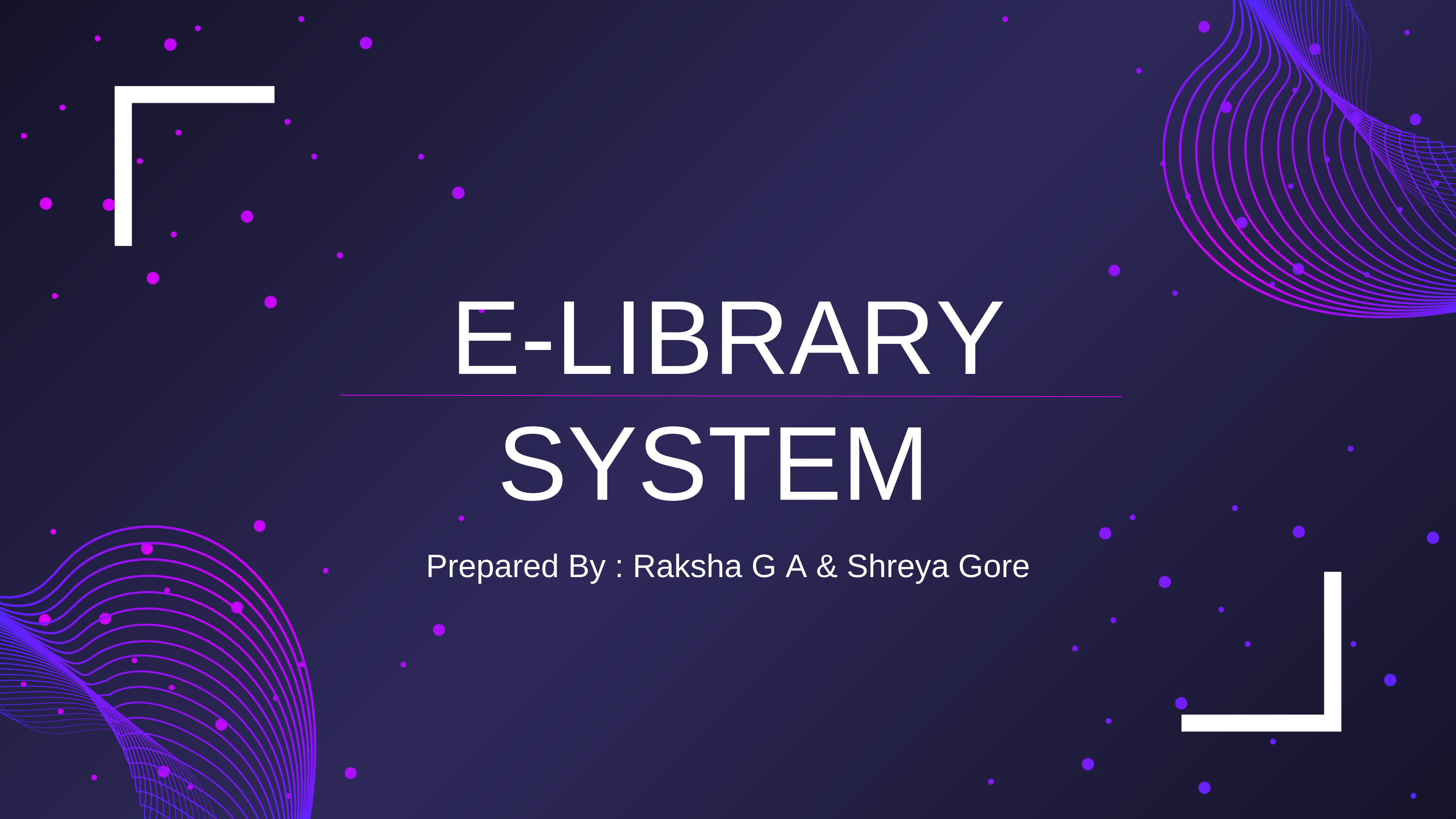




E-LIBRARY SYSTEM

Prepared By : Raksha G A & Shreya Gore

INTRODUCTION

Libraries require a simple way to manage their collection of books and members.

- This project is a console-based, menu-driven application.
- Instead of using a database, it stores information in files for persistence.

The system supports the following operations:

- Adding and deleting books
- Registering new members
- Issuing and returning books



OBJECTIVES

01

To create a simple computer-based library system.

02

To store and manage details of books and users.

03

To provide functions to issue and return books.

04

To keep a transaction history with due dates.

05

To automatically calculate fines for late returns.

06

To design a CLI (Command Line Interface) for easy use.

PROBLEM STATEMENT



- 01 Managing library operations manually is slow and error-prone.
- 02 It is hard to keep track of which books are available and which are issued.
- 03 Late return fines are difficult to calculate without proper records.
- 04 There is no simple system to store book/user data permanently.
- 05 Hence, a digital solution is needed for effective library management.

TOPICS COVERED

- Python Programming – Core language used for implementation
- Object-Oriented Programming (OOPs) – Classes for Book, User, and LibrarySystem
- File Handling – Storing and retrieving data from library.json
- JSON Module – For persistent storage of books, users, and transactions
- Logging Module – To record activities in library.log



**Python
Methods
and Class**

**Python
Data
Structure**

PYTHON



PROJECT

**Library Management
System**

TOPICS COVERED

- Exception Handling – Managing errors like duplicate IDs or missing books/users
- Date & Time Handling – Using datetime and timedelta for issue/return dates & fine calculation
- Command Line Interface (CLI) – Menu-driven system for user interaction
- Data Structures (Dict & List) – For efficient management of books, users, and transactions



Exception Handling in Python

standard
exceptions

01

try-except
block

03

raising
exceptions

05

the finally
clause

02

assertion
error

04

caching specific
exception

06



REQUIREMENTS

Functional Requirements

- Add and remove books
- Register new users
- Issue and return books with due dates
- Calculate fines for late returns
- Display available books, registered users, and active loans
- Maintain persistent storage using JSON

Non-Functional Requirements

- Easy-to-use Command Line Interface (CLI)
- Data consistency between sessions (via library.json)
- Logging of activities (library.log) for tracking
- Error handling for invalid inputs

REQUIREMENTS

Software Requirements

- Python 3.10 or 3.12 installed
- Text editor / IDE (VS Code, PyCharm, IDLE etc.)
- Command line / terminal to run the program
- Numpy and pandas library



Visual Studio



Python 3.12



FEATURES

- Book Management: Add, remove, and display all books.
- User Management: Register users and display user list.
- Issue System: Books can be issued with a fixed due date.
- Return System: Returns are recorded and fines calculated.
- Active Loans: View which books are currently issued.
- Persistent Storage: Data saved in library.json.
- Logging: Activities saved in library.log for tracking.

Library Management



Book Management

Add, remove, and display all books



Issue System

Books can be issued with a fixed due date



Return System

Returns are recorded and fines calculated



User Management

Register users and display user list



Persistent Storage

Data saved in library.json



Active Loans

View which books are currently issued



Logging

Activities saved in library.log for tracking

Active Loans

View which books are currently issued

SYSTEM ARCHITECTURE

- Modules Used: json, os, datetime, logging.
- Classes: Book – stores book details.
- User – stores user details.
- LibrarySystem – handles operations and transactions.
- Storage: All data stored in JSON file.
- Logs: System activities stored in a log file.
- Flow: User → Menu (CLI) → LibrarySystem → JSON + Logs.



SYSTEM ARCHITECTURE

- User selects an option from the menu.
- System checks data and performs the requested operation.
- Transactions (issue/return) are saved in JSON file.
- Logs are updated for each action.
- User can view current books, users, and issued records anytime.

Admin chooses -> Issue Book

Inputs -> User ID & Book ID

System checks -> User valid ? Book available ?

If valid -> Create transaction (issue_date , due_date)

Update book availability -> False

Save changes into library.json

Show output -> " Issued B101 to U001"

KEY FUNCTIONS

- `add_book()` → Adds new books to the system.
- `remove_book()` → Deletes a book if available.
- `register_user()` → Registers a new user.
- `issue_book()` → Issues book with due date.
- `give_back_book()` → Returns book and calculates fine.
- `list_books()` → Shows all books with status.
- `list_users()` → Displays registered users.
- `list_active_loans()` → Shows books currently issued.

SYSTEM DESIGN OVERVIEW

1. Modular Architecture

- System divided into classes: Book, User, LibrarySystem



2. Data Layer

- Uses library.json for storing books, users, and transactions
- Ensures persistence across sessions

3. Application Layer

- LibrarySystem acts as the controller
- Handles book issuing, returning, fine calculation, and user registration



4. Logging Layer

library.log records all operations for tracking and debugging

SYSTEM DESIGN OVERVIEW

5. User Interface Layer

- Menu-driven Command Line Interface (CLI)
- Allows user to perform operations like add, remove, issue, return



7. Error Handling

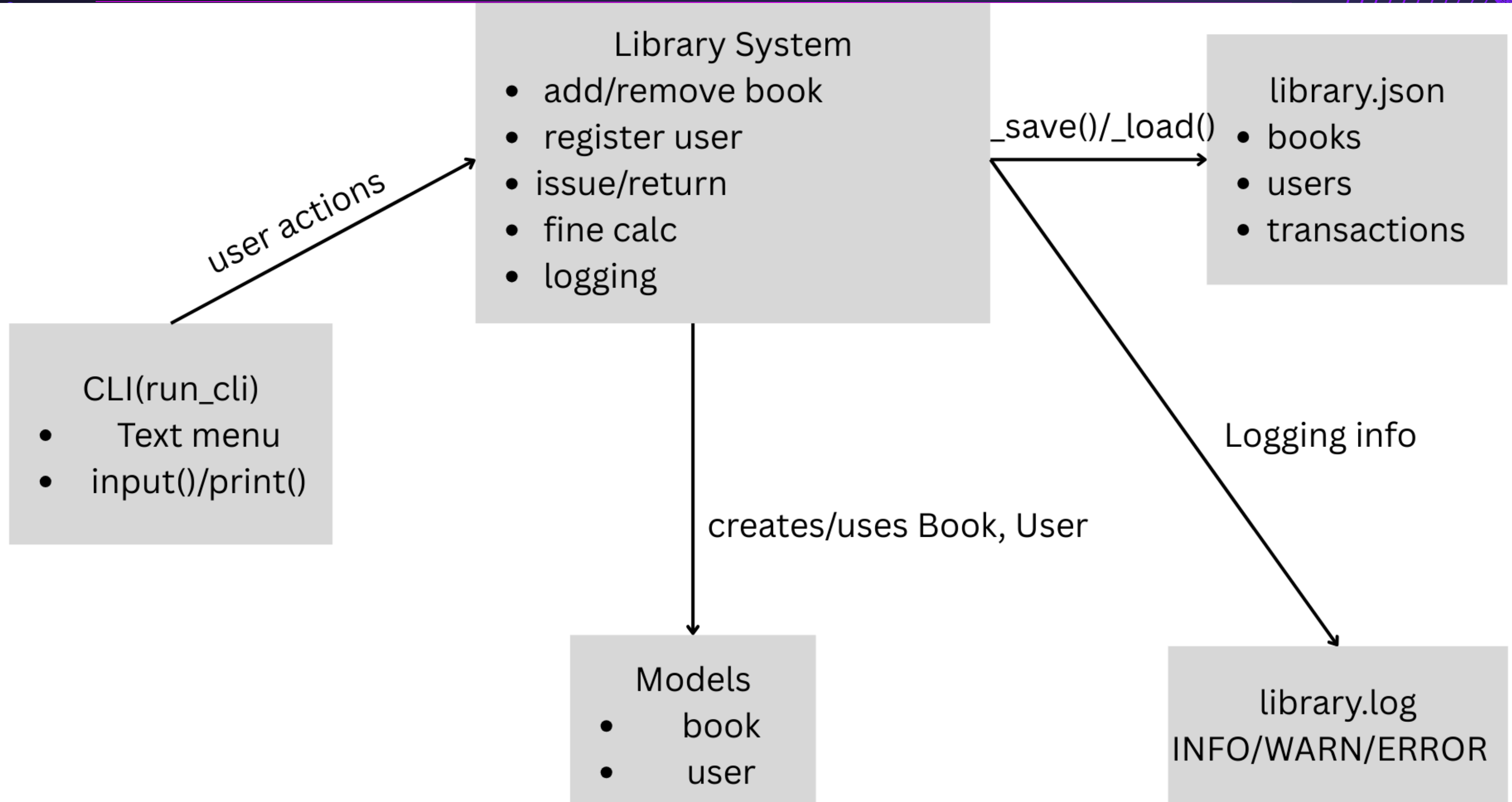
Exception handling ensures system stability during invalid inputs

6. Data Flow

Input (via CLI) → Processed by methods → Updates data (JSON + Objects) → Output shown to user

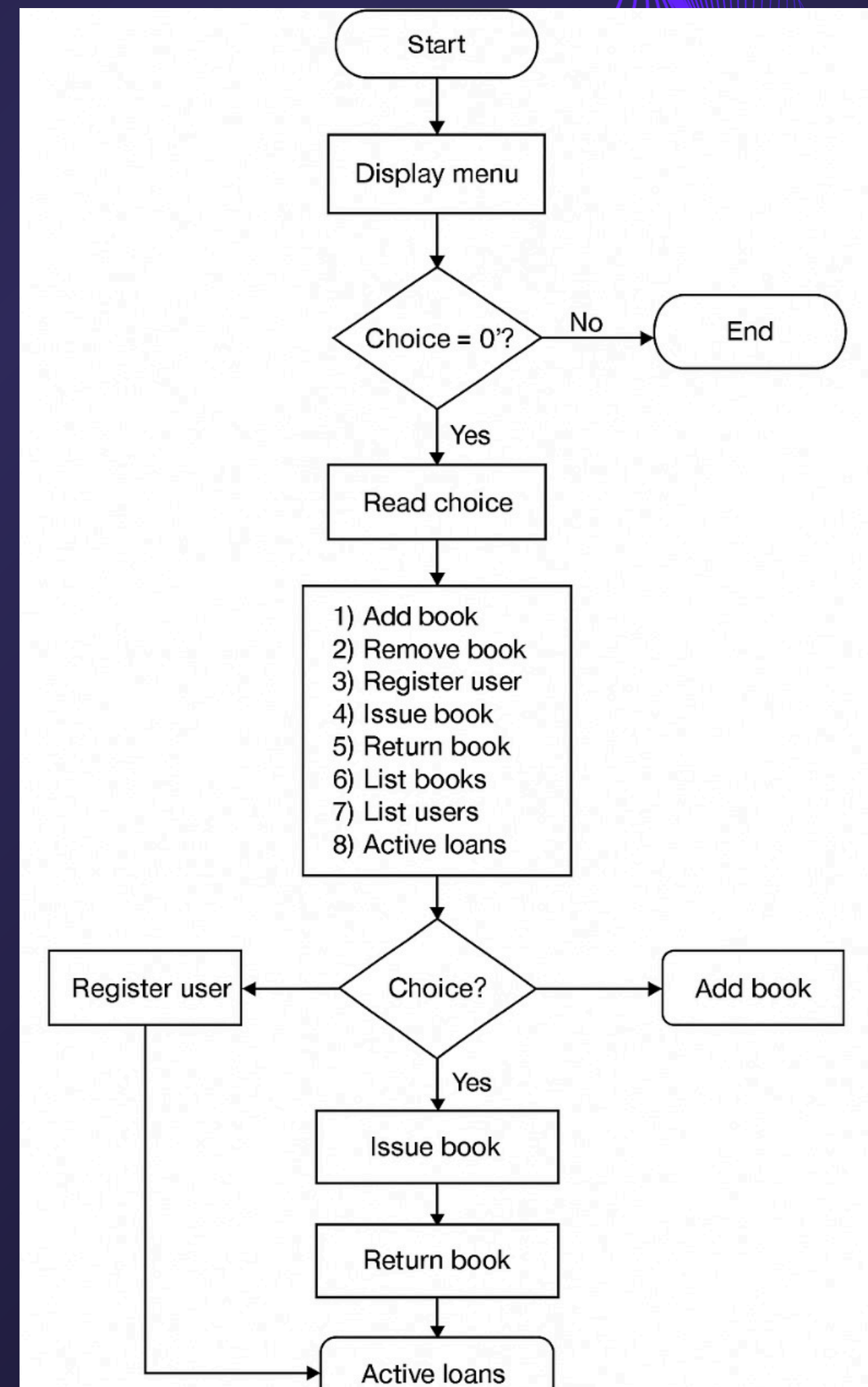
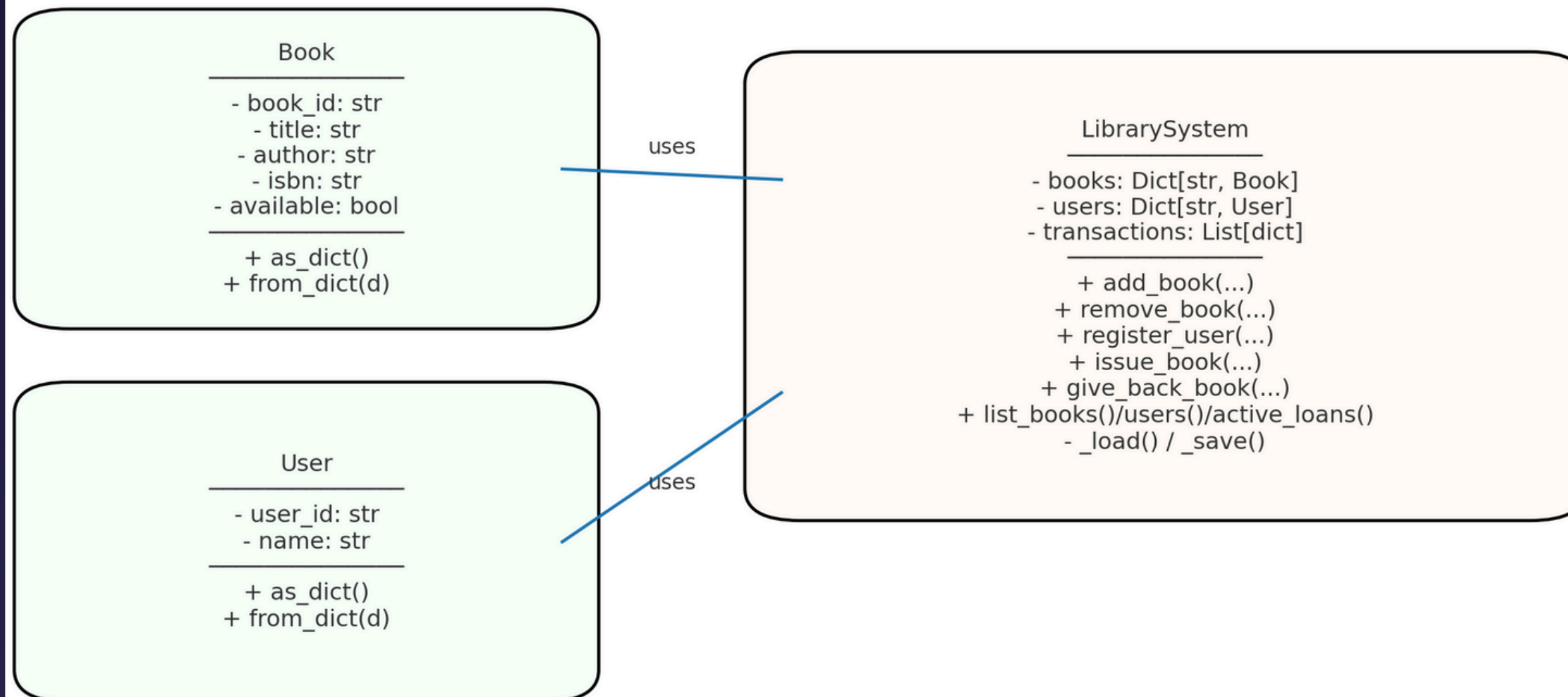


SYSTEM ARCHITECTURE



SYSTEM FLOWCHART AND CLASS DIAGRAMS

Lightweight Class Diagram



- Easy to use and manage library records.
- Prevents duplicate user or book entries.
- Tracks all transactions automatically.
- Calculates fines for late returns.
- Maintains permanent storage with JSON and logs.
- Works as a realistic mini library simulation.



CURRENT SCOPE & CONSTRAINTS



- Only CLI-based (no GUI for beginners).
- No support for book reservation or renewal.
- Limited to JSON file (no proper database).
- Works for single system (not online multi-user).
- Searching and filtering options not included.

FUTURE ENHANCEMENTS

- Build a GUI or web interface for better usability.
- Connect with database (MySQL/SQLite) for large data.
- Add search and filter features for books and users.
- Provide reservation and renewal options.
- Generate reports (monthly usage, most borrowed books).



Conclusion



The project successfully simulates a real-world library system.

Automates issue and return with fine calculation.

Demonstrates use of Python OOPs, JSON storage, logging.

Provides a strong base for future improvements.

Overall, it is a complete mini project for library automation.

The background is a deep purple color. It features two large, stylized, wavy shapes on the left and right sides, composed of many thin, concentric, curved lines in a lighter shade of purple. Scattered throughout the background are numerous small dots of varying sizes, some in a light purple and others in a slightly darker shade, creating a starry or particle-like effect.

THANK YOU